

электронную почту как *спам* и *не_спам*, и одновременно как *обычная* и *важная*. Вы наверняка пожелаали бы исключить такие прогнозы, как [*спам, важная*].

7.5. Обучение ансамбля

Фундаментальные алгоритмы, которые мы рассмотрели в главе 3, имеют свои ограничения. Из-за простоты иногда они не могут создать модель, достаточно эффективную для вашей задачи. В таких случаях можно попробовать использовать глубокие нейронные сети. Однако на практике глубокие нейронные сети требуют значительного объема размеченных данных, которых у вас может не быть. Другой способ повысить эффективность простых алгоритмов обучения — использовать **обучение ансамбля**.

Обучение ансамбля — это парадигма обучения, которая основана на обучении не одной сверхправильной модели, а большого числа моделей с низкой правильностью и объединении прогнозов, данных этими *слабыми* моделями, для получения более правильной **метамодели**.

Модели с низкой правильностью обычно обучаются **слабыми алгоритмами обучения**, которые не способны обучать сложные модели и поэтому показывают высокую скорость работы на этапах обучения и прогнозирования. Наиболее часто в роли слабого алгоритма используется алгоритм обучения дерева решений, который обычно прекращает разбивать обучающий набор после нескольких итераций. В результате получаются мелкие и не очень правильные деревья, но, как гласит идея обучения ансамбля, если деревья не идентичны и каждое дерево хотя бы немного лучше случайного угадывания, мы можем получить высокую правильность, объединив большое количество таких деревьев.

Чтобы получить окончательный прогноз для входа **x**, прогнозы всех слабых моделей объединяются с использованием некоторого метода взвешенного голосования. Конкретная форма взвешивания голосов зависит от алгоритма, но сама суть не зависит от него: если по совокупности слабые модели предсказывают, что электронное письмо является спамом, мы присваиваем образцу **x** метку *спам*.

Двумя основными методами обучения ансамблей являются **бустинг** (boosting — форсирование) и **бэггинг** (bagging — агрегирование)¹.

7.5.1. Бустинг и бэггинг

Метод бустинга заключается в использовании исходных обучающих данных и итеративного создания нескольких моделей с применением слабого алгоритма.

¹ Переводы терминов boosting и bagging неточные и не прижившиеся.

Каждая новая модель отличается от предыдущих тем, что, конструируя ее, слабый алгоритм пытается «исправить» ошибки, допускаемые предыдущими моделями. Окончательная **ансамблевая модель** представляет собой комбинацию этих многочисленных слабых моделей, построенных итеративно.

Суть бэггинга заключается в создании множества «копий» обучающих данных (каждая копия немного отличается от других) и последующем применении слабого алгоритма к каждой копии с целью получить несколько слабых моделей, а затем объединить их. Широко используемым и эффективным алгоритмом машинного обучения, основанным на идее бэггинга, является **случайный лес**.

7.5.2. Случайный лес

«Классический» алгоритм бэггинга работает следующим образом. Из имеющегося обучающего набора создается B случайных выборок $\mathcal{S}_b$ (для каждого $b = 1, \dots, B$) и на основе каждой выборки $\mathcal{S}_b$ строится модель f_b дерева решений. Чтобы получить выборку $\mathcal{S}_b$ для некоторого b , производится **выборка с заменой**. То есть сначала создается пустая выборка, а затем из обучающего набора выбирается случайный образец, и его точная копия помещается в $\mathcal{S}_b$, при этом сам образец остается в исходном обучающем наборе. Выбор данных продолжается, пока не выполнится условие $|\mathcal{S}_b| = N$.

В результате обучения получается B деревьев решений. Прогноз для нового образца $\mathbf{x}$, в случае регрессии, определяется как среднее из B прогнозов

$$y \leftarrow \hat{f}(\mathbf{x}) \stackrel{\text{def}}{=} \frac{1}{B} \sum_{b=1}^B f_b(\mathbf{x}),$$

или большинством голосов в случае классификации.

Случайный лес имеет только одно отличие от классического бэггинга. Он использует модифицированный алгоритм обучения дерева, который при каждом расщеплении в процессе обучения проверяет случайное подмножество признаков. Это делается с целью устранить корреляцию между деревьями: если один или несколько признаков имеют большую прогнозирующую способность, многие деревья будут выбирать их для расщепления данных. Это приведет к появлению в «лесу» большого числа коррелированных деревьев. Корреляция по признакам с большой прогнозирующей способностью препятствует повышению точности предсказания. Высокая эффективность ансамбля моделей объясняется тем, что хорошие модели, вероятнее всего, согласятся с одним и тем же прогнозом, а плохие — не согласятся и дадут разные прогнозы. Корреляция сделает плохие модели более склонными к согласию, что исказит картину голосования или повлияет на среднее значение.

Наиболее важными гиперпараметрами для настройки являются количество деревьев B и размер случайного подмножества признаков, которые необходимо учитывать при каждом расщеплении.

Случайный лес — один из наиболее широко используемых алгоритмов обучения ансамблей. Чем обусловлена его эффективность? Причина в том, что, используя несколько выборок из исходного набора данных, мы уменьшаем **дисперсию** конечной модели. Помните, что низкая дисперсия означает слабую предрасположенность к **переобучению**. Переобучение происходит, когда модель пытается объяснить небольшие вариации в наборе данных, потому что набор данных является лишь небольшой выборкой из всех возможных примеров явления, которое мы пытаемся смоделировать. В случае неудачного подхода к формированию обучающего набора в него могут попасть некоторые нежелательные (но неизбежные) артефакты: шум, аномальные и чрезмерно или недостаточно представительные данные. Создавая несколько случайных выборок с заменой обучающего набора, мы уменьшаем влияние этих артефактов.

7.5.3. Градиентный бустинг

Другой эффективный алгоритм обучения ансамблей, основанный на идее бустинга, — **градиентный бустинг**. Сначала рассмотрим применение градиентного бустинга в регрессии. Построение эффективной регрессионной модели мы начнем с константной модели $f = f_0$ (как мы это делали в ID3):

$$f = f_0(\mathbf{x}) \stackrel{\text{def}}{=} \frac{1}{N} \sum_{i=1}^N y_i.$$

Затем изменим метки во всех образцах $i = 1, \dots, N$ в обучающем наборе:

$$\hat{y}_i \leftarrow y_i - f(\mathbf{x}_i), \quad (7.2)$$

где $\hat{y}_i$ называется **остатком** и является новой меткой образца $\mathbf{x}_i$.

Теперь используем модифицированный обучающий набор с остатками вместо оригинальных меток, чтобы построить новую модель дерева решений, f_1 . Модель бустинга теперь определяется как $f \stackrel{\text{def}}{=} f_0 + \alpha f_1$, где α — скорость обучения (гиперпараметр).

Затем пересчитаем остатки с использованием уравнения 7.2, заменим метки в обучающих данных еще раз, обучим новую модель дерева решений f_2 , переопределим модель бустинга как $f \stackrel{\text{def}}{=} f_0 + \alpha f_1 + \alpha f_2$ и будем повторять процесс, пока не объединим предопределенное максимальное число M деревьев.

Давайте интуитивно разберемся в том, что тут происходит. Вычисляя остатки, мы определяем, насколько хорошо (или плохо) предсказывается цель каждого обучающего образца текущей моделью f . Затем мы обучим другое дерево для исправления ошибок текущей модели (именно поэтому мы используем остатки вместо фактических меток) и добавим новое дерево в существующую модель с некоторым весом α . В результате каждое новое дерево, добавленное в модель, частично исправляет ошибки, допущенные предыдущими деревьями. Процесс продолжается, пока не будет объединено максимальное количество M (еще один гиперпараметр) деревьев.

Теперь попробуем ответить на вопрос, почему этот алгоритм называется *градиентным* бустингом. В градиентном бустинге мы не вычисляем градиент, в отличие от того, что мы делали в главе 4, решая задачу линейной регрессии. Чтобы увидеть сходство между градиентным бустингом и градиентным спуском, вспомните, для чего мы вычисляли градиент в линейной регрессии: чтобы узнать направление изменения значений параметров для минимизации функции стоимости MSE. Градиент показывает направление, но не показывает, как далеко идти в этом направлении, поэтому в каждой итерации мы делали небольшой шаг, а затем вновь определяли направление. То же происходит в градиентном бустинге, только вместо непосредственного вычисления градиента мы используем его оценку в форме остатков: они показывают, как следует скорректировать модель, чтобы уменьшить ошибку (остаток).

В градиентном бустинге доступны для настройки три основных гиперпараметра: количество деревьев, скорость обучения и глубина деревьев. Все три влияют на точность модели. Глубина деревьев также влияет на скорость обучения и прогнозирования: чем меньше глубина, тем быстрее.

Можно показать, что обучение по остаткам оптимизирует общую модель f для критерия среднеквадратичной ошибки. Здесь можно заметить отличие от бэггинга: бустинг уменьшает смещение (или недообученность) вместо дисперсии. Как результат, бустинг подвержен переобучению. Однако, настраивая глубину и количество деревьев, можно в значительной степени избежать переобучения.

В задачах классификации градиентный бустинг применяется аналогично, но шаги немного отличаются. Рассмотрим случай бинарной классификации. Предположим, есть M деревьев решений регрессии. По аналогии с логистической регрессией прогноз ансамбля деревьев решений моделируется с использованием сигмоидной функции:

$$\Pr(y = 1 | \mathbf{x}, f) \stackrel{\text{def}}{=} \frac{1}{1 + e^{-f(\mathbf{x})}},$$

где $f(\mathbf{x}) \stackrel{\text{def}}{=} \sum_{m=1}^M f_m(\mathbf{x})$ и f_m — дерево регрессии.

И снова, как в логистической регрессии, при попытке найти модель f , максимизирующую $L_f = \sum_{i=1}^N \ln[\Pr(y_i = 1 | \mathbf{x}_i, f)]$, применяется принцип максимального правдоподобия. Точно так же, чтобы избежать числового переполнения, мы максимизируем сумму логарифмов правдоподобий, а не произведение правдоподобий.

Алгоритм начинает работу с начальной константной модели $f = f_0 = \frac{p}{1-p}$, где $p = \frac{1}{N} \sum_{i=1}^N y_i$. (Можно показать, что такая инициализация оптимальна для сигмоидной функции.) Затем в каждой итерации m в модель добавляется новое дерево f_m . Чтобы найти наилучшее дерево f_m , сначала вычисляется частная производная g_i текущей модели для каждого $i = 1, \dots, N$:

$$g_i = \frac{dL_f}{df},$$

где f — модель ансамблевого классификатора, построенная на предыдущей итерации $m - 1$. Чтобы вычислить g_i , нужно найти производные от $\ln[\Pr(y_i = 1 | \mathbf{x}_i, f)]$ по f для всех i . Обратите внимание, что $\ln[\Pr(y_i = 1 | \mathbf{x}_i, f)] \stackrel{\text{def}}{=} \ln\left[\frac{1}{1 + e^{-f(\mathbf{x}_i)}}\right]$. Производная по f правого члена в предыдущем уравнении равна

$$\frac{1}{e^{f(\mathbf{x}_i)} + 1}.$$

Затем выполняется преобразование обучающего набора заменой исходной метки y_i соответствующей частной производной g_i , и на основе преобразованного обучающего набора строится новое дерево f_m . Далее определяется оптимальный шаг обновления ρ_m как:

$$\rho_m \leftarrow \arg \max_{\rho} L_{f+\rho f_m}.$$

В конце итерации m мы обновляем модель ансамбля f , добавляя новое дерево f_m :

$$f \leftarrow f + \rho_m f_m.$$

Итерации продолжаются, пока не выполнится условие $m = M$, после чего обучение прекращается и в результате получается модель ансамбля f .

Градиентный бустинг является одним из самых мощных алгоритмов машинного обучения. Не только потому, что создает очень точные модели, но и потому, что способен обрабатывать огромные наборы данных с миллионами данных и признаков. Как правило, он превосходит в точности случайный лес, но из-за последовательной природы может обучаться намного медленнее.

Gradient boosting – training an ensemble based on loss gradients

Gradient boosting is another variant of the boosting concept introduced in the previous section, that is, successively training weak learners to create a strong ensemble. Gradient boosting is an extremely important topic because it forms the basis of popular machine learning algorithms such as XGBoost, which is well-known for winning Kaggle competitions.

The gradient boosting algorithm may appear a bit daunting at first. So, in the following subsections, we will cover it step by step, starting with a general overview. Then, we will see how gradient boosting is used for classification and walk through an example. Finally, after we've introduced the fundamental concepts of gradient boosting, we will take a brief look at popular implementations, such as XGBoost, and we will see how we can use gradient boosting in practice.

Comparing AdaBoost with gradient boosting

Fundamentally, gradient boosting is very similar to AdaBoost, which we discussed previously in this chapter. AdaBoost trains decision tree stumps based on errors of the previous decision tree stump. In particular, the errors are used to compute sample weights in each round as well as for computing a classifier weight for each decision tree stump when combining the individual stumps into an ensemble. We stop training once a maximum number of iterations (decision tree stumps) is reached. Like AdaBoost, gradient boosting fits decision trees in an iterative fashion using prediction errors. However, gradient boosting trees are usually deeper than decision tree stumps and have typically a maximum depth of 3 to 6 (or a maximum number of 8 to 64 leaf nodes). Also, in contrast to AdaBoost, gradient boosting does not use the prediction errors for assigning sample weights; they are used directly to form the target variable for fitting the next tree. Moreover, instead of having an individual weighting term for each tree, like in AdaBoost, gradient boosting uses a global learning rate that is the same for each tree.

As you can see, AdaBoost and gradient boosting share several similarities but differ in certain key aspects. In the following subsection, we will sketch the general outline of the gradient boosting algorithm.

Outlining the general gradient boosting algorithm

In this section, we will look at gradient boosting for classification. For simplicity, we will look at a binary classification example. Interested readers can find the generalization to the multi-class setting with logistic loss in *Section 4.6. Multiclass logistic regression and classification* of the original gradient boosting paper written by Friedman in 2001, *Greedy function approximation: A gradient boosting machine*, <https://projecteuclid.org/journals/annals-of-statistics/volume-29/issue-5/Greedy-function-approximation-A-gradient-boostingmachine/10.1214/aos/1013203451.full>.



Gradient boosting for regression

Note that the procedure behind gradient boosting is a bit more complicated than AdaBoost. We omit a simpler regression example, which was given in Friedman's paper, for brevity, but interested readers are encouraged to also consider my complementary video tutorial on gradient boosting for regression, which is available at: <https://www.youtube.com/watch?v=zblsrxc7XpM>.

In essence, gradient boosting builds a series of trees, where each tree is fit on the error—the difference between the label and the predicted value—of the previous tree. In each round, the tree ensemble improves as we are nudging each tree more in the right direction via small updates. These updates are based on a loss gradient, which is how gradient boosting got its name.

The following steps will introduce the general algorithm behind gradient boosting. After illustrating the main steps, we will dive into some of its parts in more detail and walk through a hands-on example in the next subsections.

1. Initialize a model to return a constant prediction value. For this, we use a decision tree root node; that is, a decision tree with a single leaf node. We denote the value returned by the tree as $\hat{y}$, and we find this value by minimizing a differentiable loss function L that we will define later:

$$F_0(x) = \arg \min_{\hat{y}} \sum_{i=1}^n L(y_i, \hat{y})$$

Here, n refers to the n training examples in our dataset.

2. For each tree $m = 1, \dots, M$, where M is a user-specified total number of trees, we carry out the following computations outlined in *steps 2a to 2d* below:
 - a. Compute the difference between a predicted value $F(x_i) = \hat{y}_i$ and the class label y_i . This value is sometimes called the *pseudo-response* or *pseudo-residual*. More formally, we can write this pseudo-residual as the negative gradient of the loss function with respect to the predicted values:

$$r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F(x)=F_{m-1}(x)} \quad \text{for } i = 1, \dots, n$$

Note that in the notation above $F(x)$ is the prediction of the previous tree, $F_{m-1}(x)$. So, in the first round, this refers to the constant value from the tree (single leaf node) from step 1.

- b. Fit a tree to the pseudo-residuals r_{im} . We use the notation R_{jm} to denote the $j = 1 \dots J_m$ leaf nodes of the resulting tree in iteration m .

- c. For each leaf node R_{jm} , we compute the following output value:

$$\gamma_{jm} = \arg \min_{\gamma} \sum_{x_i \in R_{jm}} L(y_i, F_{m-1}(x_i) + \gamma)$$

In the next subsection, we will dive deeper into how this γ_{jm} is computed by minimizing the loss function. At this point, we can already note that leaf nodes R_{jm} may contain more than one training example, hence the summation.

- d. Update the model by adding the output values γ_m to the previous tree:

$$F_m(x) = F_{m-1}(x) + \eta \gamma_m$$

However, instead of adding the full predicted values of the current tree γ_m to the previous tree F_{m-1} , we scale γ_m by a learning rate η , which is typically a small value between 0.01 and 1. In other words, we update the model incrementally by taking small steps, which helps avoid overfitting.

Now, after looking at the general structure of gradient boosting, we will adopt these mechanics to look at gradient boosting for classification.

Explaining the gradient boosting algorithm for classification

In this subsection, we will go over the details for implementing the gradient boosting algorithm for binary classification. In this context, we will be using the logistic loss function that we introduced for logistic regression in *Chapter 3, A Tour of Machine Learning Classifiers Using Scikit-Learn*. For a single training example, we can specify the logistic loss as follows:

$$L_i = -y_i \log p_i + (1 - y_i) \log(1 - p_i)$$

In *Chapter 3*, we also introduced the log(odds):

$$\hat{y} = \log(\text{odds}) = \log\left(\frac{p}{1-p}\right)$$

For reasons that will make sense later, we will use these log(odds) to rewrite the logistic function as follows (omitting intermediate steps here):

$$L_i = \log(1 + e^{\hat{y}_i}) - y_i \hat{y}_i$$

Now, we can define the partial derivative of the loss function with respect to these log(odds), $\hat{y}$. The derivative of this loss function with respect to the log(odds) is:

$$\frac{\partial L_i}{\partial \hat{y}_i} = \frac{e^{\hat{y}_i}}{1 + e^{\hat{y}_i}} - y_i = p_i - y_i$$

After specifying these mathematical definitions, let us now revisit the general gradient boosting *steps 1 to 2d* from the previous section and reformulate them for this binary classification scenario.

1. Create a root node that minimizes the logistic loss. It turns out that the loss is minimized if the root node returns the $\log(\text{odds})$, $\hat{y}$.
2. For each tree $m = 1, \dots, M$, where M is a user-specified number of total trees, we carry out the following computations outlined in *steps 2a to 2d*:
 - a. We convert the $\log(\text{odds})$ into a probability using the familiar logistic function that we used in logistic regression (in *Chapter 3*):

$$p = \frac{1}{1 + e^{-\hat{y}}}$$

Then, we compute the pseudo-residual, which is the negative partial derivative of the loss with respect to the $\log(\text{odds})$, which turns out to be the difference between the class label and the predicted probability:

$$-\frac{\partial L_i}{\partial \hat{y}_i} = y_i - p_i$$

- b. Fit a new tree to the pseudo-residuals.
- c. For each leaf node R_{jm} , compute a value γ_{jm} that minimizes the logistic loss function. This includes a summarization step for dealing with leaf nodes that contain multiple training examples:

$$\begin{aligned} \gamma_{jm} &= \arg \min_{\gamma} \sum_{x_i \in R_{jm}} L(y_i, F_{m-1}(x_i) + \gamma) \\ &= \log(1 + e^{\hat{y}_i + \gamma}) - y_i(\hat{y}_i + \gamma) \end{aligned}$$

Skipping over intermediate mathematical details, this results in the following:

$$\gamma_{jm} = \frac{\sum_i y_i - p_i}{\sum_i p_i(1 - p_i)}$$

Note that the summation here is only over the examples at the node corresponding to the leaf node R_{jm} and not the complete training set.

- d. Update the model by adding the gamma value from *step 2c* with learning rate η :

$$F_m(x) = F_{m-1}(x) + \eta \gamma_m$$



Outputting log(odds) vs probabilities

Why do the trees return log(odds) values and not probabilities? This is because we cannot just add up probability values and arrive at a meaningful result. (So, technically speaking, gradient boosting for classification uses regression trees.)

In this section, we adopted the general gradient boosting algorithm and specified it for binary classification, for instance, by replacing the generic loss function with the logistic loss and the predicted values with the log(odds). However, many of the individual steps may still seem very abstract, and in the next section, we will apply these steps to a concrete example.

Illustrating gradient boosting for classification

The previous two subsections went over the condensed mathematical details of the gradient boosting algorithm for binary classification. To make these concepts clearer, let's apply it to a small toy example, that is, a training dataset of the following three examples shown in *Figure 7.12*:

	Feature x_1	Feature x_2	Class label y
1	1.12	1.4	1
2	2.45	2.1	0
3	3.54	1.2	1

Figure 7.12: Toy dataset for explaining gradient boosting

Let's start with *step 1*, constructing the root node and computing the log(odds), and *step 2a*, converting the log(odds) into class-membership probabilities and computing the pseudo-residuals. Note that based on what we have learned in *Chapter 3*, the odds can be computed as the number of successes divided by the number of failures. Here, we regard label 1 as success and label 0 as failure, so the odds are computed as: odds = 2/1. Carrying out steps 1 and 2a, we get the following results shown in *Figure 7.13*:

	Feature x_1	Feature x_2	Class label y	Step 1: $\hat{y} = \log(\text{odds})$	Step 2a: $p = \frac{1}{1 + e^{-\hat{y}}}$	Step 2a: $r = y - p$
1	1.12	1.4	1	0.69	0.67	0.33
2	2.45	2.1	0	0.69	0.67	-0.67
3	3.54	1.2	1	0.69	0.67	0.33

Figure 7.13: Results from the first round of applying step 1 and step 2a

Next, in *step 2b*, we fit a new tree on the pseudo-residuals r . Then, in *step 2c*, we compute the output values, γ , for this tree as shown in *Figure 7.14*:

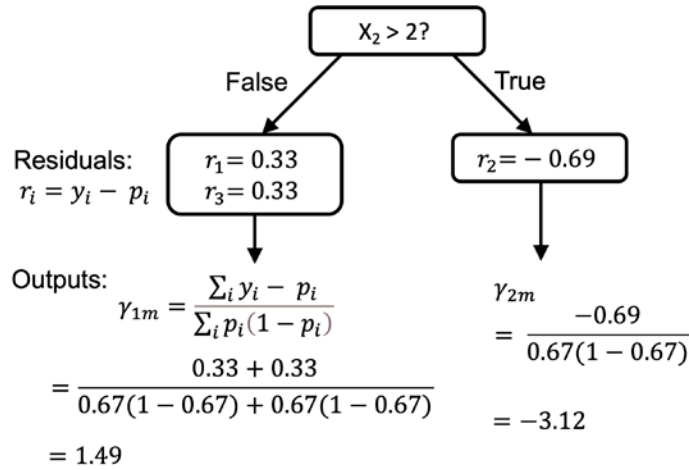


Figure 7.14: An illustration of steps 2b and 2c, which fits a tree to the residuals and computes the output values for each leaf node

(Note that we artificially limit the tree to have only two leaf nodes, which helps illustrate what happens if a leaf node contains more than one example.)

Then, in the final *step 2d*, we update the previous model and the current model. Assuming a learning rate of $\eta = 0.1$, the resulting prediction for the first training example is shown in *Figure 7.15*:

	Feature x_1	Feature x_2	Class label y
➔ 1	1.12	1.4	1
2	2.45	2.1	0
3	3.54	1.2	1

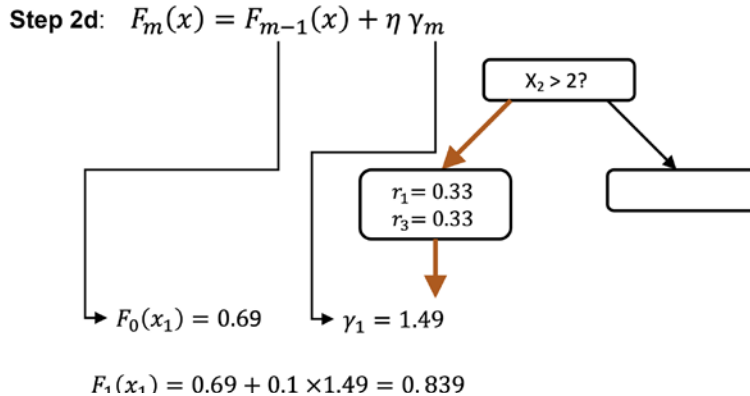


Figure 7.15: The update of the previous model shown in the context of the first training example

Now that we have completed *steps 2a* to *2d* of the first round, $m = 1$, we can proceed to execute *steps 2a* to *2d* for the second round, $m = 2$. In the second round, we use the $\log(\text{odds})$ returned by the updated model, for example, $F_1(x_1) = 0.839$, as input to *step 2A*. The new values we obtain in the second round are shown in *Figure 7.16*:

x_1	x_2	y	Step 1: $F_0(x) = \hat{y}$ $= \log(\text{odds})$	Step 2a: $p = \frac{1}{1 + e^{-\hat{y}}}$	Step 2a: $r = y - p$	New $\log(\text{odds})$ $\hat{y} = F_1(x)$	Step 2a: p	Step 2a: r	
1	1.12	1.4	1	0.69	0.67	0.33	0.839	0.698	0.302
2	2.45	2.1	0	0.69	0.67	-0.67	0.378	0.593	-0.593
3	3.54	1.2	1	0.69	0.67	0.33	0.839	0.698	0.302

Round $m = 1$

Round $m = 2$

Figure 7.16: Values from the second round next to the values from the first round

We can already see that the predicted probabilities are higher for the positive class and lower for the negative class. Consequently, the residuals are getting smaller, too. Note that the process of *steps 2a* to *2d* is repeated until we have fit M trees or the residuals are smaller than a user-specified threshold value. Then, once the gradient boosting algorithm has completed, we can use it to predict the class labels by thresholding the probability values of the final model, $F_M(x)$ at 0.5, like logistic regression in *Chapter 3*. However, in contrast to logistic regression, gradient boosting consists of multiple trees and produces nonlinear decision boundaries. In the next section, we will look at how gradient boosting looks in action.

Using XGBoost

After covering the nitty-gritty details behind gradient boosting, let's finally look at how we can use gradient boosting code implementations.

In scikit-learn, gradient boosting is implemented as `sklearn.ensemble.GradientBoostingClassifier` (see <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingClassifier.html> for more details). It is important to note that gradient boosting is a sequential process that can be slow to train. However, in recent years a more popular implementation of gradient boosting has emerged, namely, XGBoost.

XGBoost proposed several tricks and approximations that speed up the training process substantially. Hence, the name XGBoost, which stands for extreme gradient boosting. Moreover, these approximations and tricks result in very good predictive performances. In fact, XGBoost gained popularity as it has been the winning solution for many Kaggle competitions.

Next to XGBoost, there are also other popular implementations of gradient boosting, for example, LightGBM and CatBoost. Inspired by LightGBM, scikit-learn now also implements a `HistGradientBoostingClassifier`, which is more performant than the original gradient boosting classifier (`GradientBoostingClassifier`).

You can find more details about these methods via the resources below:

- **XGBoost**: <https://xgboost.readthedocs.io/en/stable/>
- **LightGBM**: <https://lightgbm.readthedocs.io/en/latest/>
- **CatBoost**: <https://catboost.ai>
- **HistGradientBoostingClassifier**: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.HistGradientBoostingClassifier.html>

However, since XGBoost is still among the most popular gradient boosting implementations, we will see how we can use it in practice. First, we need to install it, for example via pip:

```
pip install xgboost
```

Installing XGBoost



For this chapter, we used XGBoost version 1.5.0, which can be installed via:

```
pip install XGBoost==1.5.0
```

You can find more information about the installation details at <https://xgboost.readthedocs.io/en/stable/install.html>

Fortunately, XGBoost's `XGBClassifier` follows the scikit-learn API. So, using it is relatively straightforward:

```
>>> import xgboost as xgb
>>> model = xgb.XGBClassifier(n_estimators=1000, learning_rate=0.01,
...                           max_depth=4, random_state=1,
...                           use_label_encoder=False)

>>> gbm = model.fit(X_train, y_train)
>>> y_train_pred = gbm.predict(X_train)
>>> y_test_pred = gbm.predict(X_test)

>>> gbm_train = accuracy_score(y_train, y_train_pred)
>>> gbm_test = accuracy_score(y_test, y_test_pred)
>>> print(f'XGboost train/test accuracies '
...       f'{gbm_train:.3f}/{gbm_test:.3f}')
XGboost train/test accuracies 0.968/0.917
```

Here, we fit the gradient boosting classifier with 1,000 trees (rounds) and a learning rate of 0.01. Typically, a learning rate between 0.01 and 0.1 is recommended. However, remember that the learning rate is used for scaling the predictions from the individual rounds. So, intuitively, the lower the learning rate, the more estimators are required to achieve accurate predictions.